

Full-Stack Benchmarks of a Blackwell-Generation AI Cluster: Compute, Interconnect, and End-to-End Training

Shaohao Chen*, Lauren Milechin, and Christopher N. Hill
*Office of Research Computing and Data (ORCD),
Massachusetts Institute of Technology (MIT), Cambridge, MA, USA*
*shaohao@mit.edu

Abstract—We present a layered performance characterization of the MGHPCC AI Computing Resource (AICR), a newly deployed NVIDIA Blackwell cluster with B200 and RTX PRO 6000 nodes. We benchmark three layers of the system stack: GPU compute (general matrix multiplication, GEMM), the NVIDIA Collective Communications Library (NCCL) collective interconnect over NVLink, PCIe, and NDR InfiniBand, and end-to-end large-language-model training (Megatron-LM), classifying each result as bound by a hardware ceiling or a software limit. B200 compute and intra-node communication run close to their hardware ceilings; the recoverable performance is inter-node, where a hardware bidirectional wall caps collectives but SHARP in-network reduction bypasses it for a measured $2.2\times$ All-Reduce speedup, while All-to-All remains an algorithm limit. The non-NVLink RTX PRO 6000 nodes perform as expected for their PCIe-based design, which suits them to single- or few-GPU workloads. Training scales efficiently at a stable fraction of the raw GEMM rate on both GPU types. In multi-node Megatron-LM data-parallel training, intra-node weak scaling is near-perfect and the inter-node gradient-synchronization cost falls as nearly $1/k$ with the number of GPUs per node k , so packing GPUs densely per node minimizes communication overhead. This work contributes detailed Blackwell GPU and interconnect benchmarks for the HPC and AI communities and distills them into practical guidelines for choosing a parallelism strategy.

Index Terms—GPU benchmarking, Blackwell, NCCL, SHARP, InfiniBand, collective communication, LLM training, model FLOPs utilization

I. INTRODUCTION

The Artificial Intelligence Computing Resource (AICR) [1] is a new collaborative compute resource at the Massachusetts Green High Performance Computing Center (MGHPCC), delivering the advanced computing and data infrastructure needed to support AI-driven research and innovation across the six member universities (Boston University, Harvard, MIT, Northeastern, the University of Massachusetts, and Yale) and their partners, in collaboration with the Massachusetts AI Hub. AICR supports workloads that range from AI training and inference to broad scientific research disciplines, such as nuclear physics, molecular dynamics, and neuroscience.

AICR is built on NVIDIA’s Blackwell GPUs [2]. It comprises 31 B200 nodes (248 GPUs total) and 19 Blackwell RTX PRO 6000 nodes (152 GPUs total). Within a B200 node the eight GPUs are linked by NVLink, the RTX PRO

6000 nodes use PCIe, and all nodes are connected by an NDR InfiniBand network (NVIDIA Quantum-2) [3]. Because Blackwell is newly deployed, few studies have so far looked at how it behaves across the system stack, and AI researchers and HPC users need guidance on parallelism strategy and realistic performance expectations before scaling up.

Existing studies offer only partial guidance for such a system. GPU microbenchmarking has largely tracked the datacenter accelerator generations: a vendor architecture whitepaper characterized the Hopper generation (H100, H200) for dense GEMM and transformer workloads [4]. Beyond the vendor architecture brief [2], an early independent microbenchmark study of Blackwell [5] dissected its new tensor cores and memory subsystem at the instruction level, though at the compute layer only. The roofline model [6] remains the standard lens for separating compute-bound from bandwidth-bound behavior, the same hardware-versus-software classification we apply per layer. On the interconnect side, prior work has evaluated GPU fabrics (PCIe, NVLink, NVSwitch, and GPUDirect) and their effect on collective performance [7], characterized NCCL collective algorithms and topology sensitivity [8], and analyzed in-network reduction with the Scalable Hierarchical Aggregation and Reduction Protocol (SHARP) on InfiniBand fabrics [9].

For end-to-end training, MLPerf Training [10] is the standard cross-vendor benchmark, and its Blackwell results to date are vendor and cloud submissions at thousand-GPU scale rather than independent studies. Model FLOPs Utilization (MFU) was popularized as a cross-generation efficiency metric for large transformer pretraining [11], and Megatron-LM [12] established the standard tensor-, pipeline-, and data-parallel large-language-model (LLM) training stack on PyTorch [13], with its weak-scaling behavior benchmarked at scale on an earlier Ampere generation [14].

These efforts each target a single layer or a prior GPU generation. To our knowledge, no study yet provides a layered, hardware-versus-software-classified characterization of the Blackwell B200 or the RTX PRO 6000 spanning compute, interconnect, and end-to-end training, nor a side-by-side of a NVLink-equipped datacenter part against a non-NVLink workstation part on the same fabric and software stack, the gap this paper fills.

The remainder of the paper describes the testbed and methodology, then works up the stack from GPU compute to NCCL collectives to end-to-end Megatron-LM training, and closes with discussions and conclusions.

II. TESTBED AND METHODOLOGY

Table I summarizes the two GPU node types. Each B200 node pairs eight B200 GPUs (193 GB HBM3e each) over a fifth-generation NVLink and NVSwitch fabric providing about 900 GB/s per GPU per direction; each RTX PRO 6000 node pairs eight workstation-class Blackwell GPUs (96 GB GDDR7 each) connected only by PCIe Gen5. B200 nodes carry eight NDR400 InfiniBand NICs (400 Gb/s each, one per GPU), rail-optimized [15] and supporting in-network SHARP reduction, whereas each RTX PRO 6000 node has a single InfiniBand card. The B200 nodes are liquid-cooled and the RTX PRO 6000 nodes air-cooled.

TABLE I
AICR GPU NODE CONFIGURATION

	B200 node ($\times 31$)	RTX PRO 6000 node ($\times 19$)
GPUs/node	8 $\times$ B200	8 $\times$ RTX PRO 6000
GPU memory	193 GB HBM3e	96 GB GDDR7
Intra-node	NVLink 5.0 + NVSwitch	PCIe Gen5 + Infinity Fabric (IF), NPS=4
NICs	8 $\times$ NDR400, SHARP	NDR IB (per node)
CPU	2 $\times$ EPYC 9575F (128 cores)	2 $\times$ EPYC 9575F (128 cores)
Host mem	2.25 TB	2.25 TB
Cooling	liquid	air

GPU compute is benchmarked with `gpu-fryer` [16], a compute microbenchmark run from a container that drives a tight large-tile cuBLAS GEMM loop at various floating-point precisions. We take the steady-state plateau of the loop as the per-GPU mean over the eight GPUs in a node, and compare it against the *dense* Tensor-Core peak (no sparsity), the vendor ceiling derived from core count, operations per cycle, and boost clock.

Interconnect bandwidth uses the NCCL performance tests [17], compiled against the NCCL library [8] in the NVHPC 26.3 toolchain, which also bundles (via HPC-X) the `nccl_rdma_sharp_plugin` that provides the GPUDirect-RDMA (remote direct memory access) and SHARP transports. We sweep message sizes up to 16 GB and report the converged large-message value, since that is where the fabric, not launch latency, binds; each point is averaged over repeated iterations after a warm-up pass, and all runs report zero mismatched elements in the built-in correctness check. We report NCCL bus bandwidth (*busbw*) rather than algorithm bandwidth (*algbw*) [17]; *busbw* applies the per-collective multiplier that converts user-visible throughput into bytes-on-the-wire per GPU per direction and is therefore the quantity directly comparable to a link ceiling.

End-to-end AI training uses Megatron-LM on PyTorch, run from a container; its gradient-sync and parallelism collectives run over NCCL. We report training throughput as the last-iteration TFLOP/s/GPU (analytical model FLOPs over measured step time) and Model FLOPs Utilization, $MFU = F/(t \cdot P)$, where F is the analytical model FLOPs per step, t

TABLE II
B200 DENSE GEMM: ACHIEVED VERSUS THEORETICAL TENSOR-CORE PEAK (TFLOP/s, PER-GPU). THE DENSE PEAK IS THE VENDOR SPEC [2].

Precision	Achieved	Dense peak	% of peak
FP32 (TF32)	768	1,125	68
BF16	1,493	2,250	66
FP8	4,103	4,500	91

the measured step time, and P the hardware peak. Multi-node efficiency is taken against the matching single-node baseline at the same GPUs per node so that it isolates only the added inter-node cost.

III. GPU COMPUTE: GEMM THROUGHPUT

We measure dense GEMM throughput running FP32, BF16, and FP8 on B200 and RTX PRO 6000 nodes. Fig. 1 reports the per-GPU mean across the eight GPUs, with H200 and L40S reference values from the MIT Engaging computing cluster for context.

Across GPUs, the B200 delivers about $2.1\times$ the throughput of an H200 in FP32 and BF16, widening to $2.8\times$ at FP8 because Blackwell’s fifth-generation Tensor Cores raise the FP8 ceiling disproportionately. Despite sharing the Blackwell architecture, the B200 outperforms the RTX PRO 6000 by $3.6\text{--}3.7\times$ in FP32/BF16 and $4.7\times$ in FP8: the B200 uses the large data-center GB200 die with HBM3e, while the RTX PRO 6000 uses a smaller workstation-class die with GDDR7. The RTX PRO 6000 in turn roughly doubles the previous-generation L40S workstation part ($2.1\times$ in FP32/BF16). All nodes passed health checks with no ECC errors, no throttling, and consistent intra-node uniformity.

Table II places the B200 numbers against the dense Tensor-Core peak. The FP8 path reaches 91% of peak, an excellent result, while FP32 (which cuBLAS runs as TF32 on the Tensor Cores [18]) and BF16 sit at 68% and 66%, the lower fractions typical of the wider-datatype Tensor-Core paths. The per-precision scaling FP32:BF16:FP8 is 1:1.94:5.34, and the same data-type scaling law (FP32:BF16 = 1:1.94) appears on H200, confirming healthy Tensor Cores rather than a configuration artifact.

IV. INTERCONNECT: NCCL COLLECTIVES

A. Intra-node (NVLink / NVSwitch)

The left half of Table III reports the intra-node results of B200 nodes: *busbw* against the NVLink bound. The NVSwitch fabric is all-to-all, providing the full 900 GB/s between any two of the eight GPUs, and all collectives reach 74–93% of that bound. The remaining gap from 100% comes from startup latency folded into the timing and from algorithm overhead, which sorts the collectives into three tiers by mechanism. In-network reduction is best: All-Reduce reaches 93% because NCCL runs it with the NVLink SHARP (NVLS) algorithm [19], whose in-switch reduction makes the operation nearly single-pass. Ring/chain algorithms (Reduce, Broadcast, Reduce-Scatter, and All-Gather) sit at 76–78%, limited by

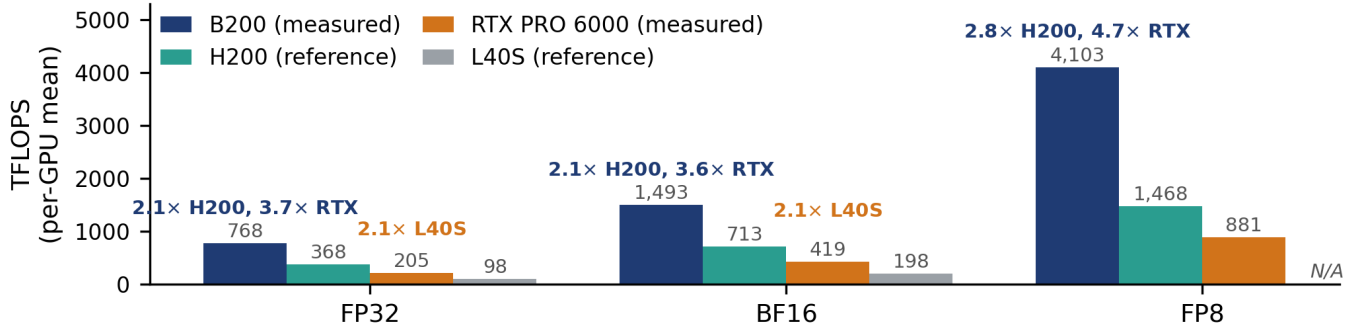


Fig. 1. Dense GEMM throughput (TFLOPS, per-GPU mean) across precisions. B200 and RTX PRO 6000 are measured; H200 and L40S are reference values. Bold labels give the B200 speedup over H200 and over RTX PRO 6000, and the RTX PRO 6000 speedup over L40S (L40S has no FP8 path).

pipeline fill and drain. Point-to-point patterns (Send-Recv, and All-to-All, which NCCL builds from many Send-Recevs) trail at 74–75%, engaging fewer NVLink channels than a full collective. Intra-node communication is therefore close to its hardware ceiling.

TABLE III
NCCL BUS BANDWIDTH ON B200 AT 16 GB (BUSBW, GB/s):
INTRA-NODE (LEFT, 1 NODE) VERSUS INTER-NODE (RIGHT, 2 NODES)

Collective	Intra-node (NVLink)		Inter-node (NDR IB)	
	busbw	% max	busbw	% max
Send-Recv	666	74	26.6	~100
Reduce	701	78	201	94
Broadcast	691	77	202	94
Reduce-Scatter	695	77	218	~100
All-Gather	684	76	218	~100
All-Reduce	841	93	170	79
All-to-All	675	75	39.8	19

B. Inter-node (NDR InfiniBand)

The right half of Table III reports the inter-node results (two nodes with NDR InfiniBand), compared against the GPUDirect RDMA (GDRDMA) aggregate ceiling. Although each NDR NIC is rated at 50 GB/s per direction, a B200 GPU reaches its NIC through a single 16-lane PCIe Gen5 port whose DMA engine [20] has a fixed ≈ 53.5 GB/s HBM budget shared between transmit and receive; under simultaneous bidirectional traffic each direction collapses to half of that, about 26.7 GB/s, so the per-node aggregate for ring-style collectives is $8 \times 26.7 \approx 214$ GB/s per direction. Per GPU, this 26.7 GB/s inter-node budget is more than an order of magnitude below the 900 GB/s NVLink bound, which is why packing GPUs densely per node pays off. Send-Recv measures 26.6 GB/s, essentially 100% of the per-pair bidirectional limit, a silicon-level wall that no NCCL tuning can overcome. All-Gather and Reduce-Scatter saturate the 214 GB/s aggregate; Reduce and Broadcast reach 94%.

Two patterns fall well short and are algorithm-limited, not hardware-limited. Plain-ring All-Reduce reaches only 79% (170 GB/s), even though its two phases, Reduce-Scatter and

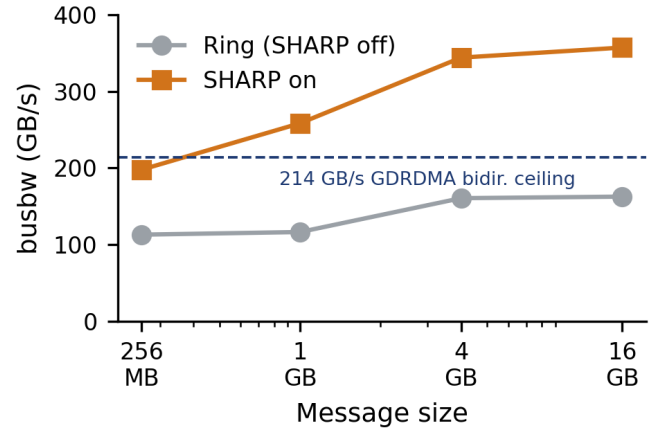


Fig. 2. Inter-node All-Reduce bus bandwidth versus message size, Ring versus SHARP (8 GPUs/node, 2 nodes).

All-Gather [21], each saturate the 214 GB/s aggregate alone. The cause is not the doubled traffic (busbw divides that out) but running the two phases in sequence: the ring fills and drains twice and pays the phase-transition latency, overhead busbw does not normalize, leaving All-Reduce below the single-pass All-Gather. SHARP removes exactly this penalty by collapsing the two passes into one in-switch reduction (Section IV-C).

All-to-All collapses to 19%: NCCL implements it as N^2 point-to-point transfers not pipelined across all NICs [8], whereas the same fabric delivers 218 GB/s for All-Gather. Because the limit is algorithmic, not physical, a CUDA-aware MPI All-to-All (whose pairwise-exchange and Bruck schedules [22], [23] pipeline inter-node transfers better than NCCL’s) or an NVSHMEM-based dispatch [24] could recover part of the gap, though neither can exceed the GDRDMA wall; we did not benchmark these.

C. SHARP in-network reduction

SHARP performs the reduction arithmetic inside the InfiniBand switch, turning All-Reduce into a single pass: each GPU sends its data up once and receives the reduced result

down once, halving wire and PCIe traffic and bypassing the 214 GB/s bidirectional wall. Fig. 2 shows a clean comparison of Ring (SHARP off) versus SHARP (on) at eight GPUs per node across two nodes. SHARP wins above about 4 MB and grows with message size, peaking at 357 GB/s versus 163 GB/s for Ring at 16 GB, a 2.2 $\times$ speedup, with validation clean. The benefit requires the full complement of eight NICs per node; at two GPUs per node SHARP is about 9% slower than Ring because too few NICs are engaged to bottleneck the aggregate. Enabling SHARP requires forcing NCCL’s CollNet path and restricting NCCL to the SHARP-connected NICs. Both are configured entirely through NCCL environment variables, such as `NCCL_COLLNET_ENABLE` and `NCCL_ALGO`, so no application code changes are needed; Megatron-LM and PyTorch use NCCL underneath.

D. RTX PRO 6000 collectives: no NVLink

The RTX PRO 6000 nodes have no NVLink, so GPU-to-GPU communication uses PCIe Gen5 peer-to-peer, relayed through the AMD EPYC processor whose I/O die is an Infinity Fabric mesh. In the platform’s NUMA-per-socket mode NPS=4, each socket exposes four sub-NUMA dies and no GPU pair shares a PCIe switch, so *every* intra-node transfer is routed through the CPU and crosses IF between dies. This produces three regimes, each with a different binding constraint, summarized in Table IV. With a single active pair (two GPUs, one IF hop), Send-Recv reaches 37.4 GB/s per direction, bounded by the GPU’s PCIe DMA bidirectional budget, 59% of the 16-lane PCIe Gen5 link rate (≈ 63 GB/s per direction [25]). Adding GPUs is severe for symmetric collectives: at four GPUs across four dies, all of Send-Recv, All-Reduce, All-Gather, Reduce-Scatter, and All-to-All collapse to a ≈ 13 GB/s floor as concurrent bidirectional traffic saturates the on-package fabric (Fig. 3). The exceptions are root-anchored collectives, Scatter (50.6 GB/s) and Gather (39.2 GB/s), whose traffic is unidirectional at the root and therefore escapes the bidir DMA split. Relative to B200’s NVLink, RTX PRO 6000 symmetric collectives are 18–64 $\times$ slower. Inter-node, a single GPU per node reaches 24.7 GB/s on Send-Recv, GDRDMA-bidir limited by the same PCIe DMA mechanism as B200 (26.6 GB/s) and thus nearly identical.

TABLE IV
RTX PRO 6000 NCCL BUSBW (GB/s, 16 GB MESSAGE)

Configuration	Binding limit	busbw
1 node, 2 GPU, Send-Recv	PCIe DMA bidir	37.4
1 node, 4 GPU, symmetric coll.	IF 4-die saturation	~ 13
1 node, 4 GPU, Scatter/Gather	root unidir (IF)	50.6 / 39.2
2 node, 1 GPU/node, Send-Recv	GDRDMA bidir	24.7

E. Implications for parallelism

Within a B200 node the NVLink/NVSwitch fabric puts every collective near its hardware ceiling, so parallelism is essentially unconstrained inside a node: even the bandwidth-hungriest pattern, tensor parallelism’s per-transformer-layer

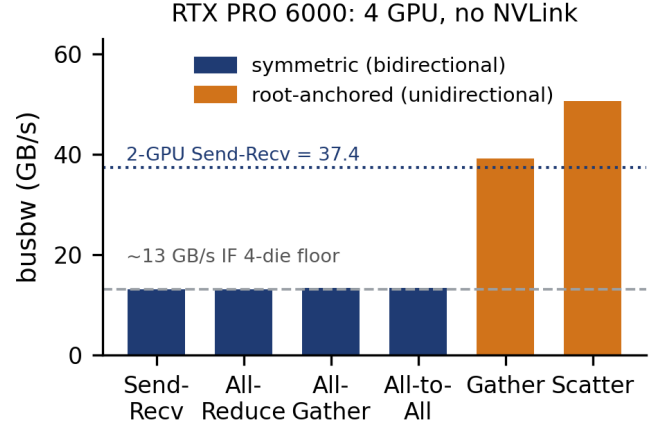


Fig. 3. RTX PRO 6000 four-GPU NCCL bus bandwidth by collective. The dashed line marks the ≈ 13 GB/s Infinity Fabric four-die floor and the dotted line the two-GPU single-hop Send-Recv.

activation All-Reduce, runs at 841 GB/s. Across nodes the dominant collective decides feasibility: data-parallel All-Reduce is the primary beneficiary of SHARP (357 GB/s), sequence-parallel All-Gather and Reduce-Scatter reach 218 GB/s each, pipeline-parallel Send-Recv is bounded by the 26.6 GB/s GDRDMA wall, and MoE All-to-All is the weakest. On RTX PRO 6000 all of these patterns are viable only up to two GPUs (Section IV-D). Section VI later distills these bandwidths into operational guidance (Table VIII).

V. END-TO-END TRAINING: MEGATRON-LM

We train a 1.32 B-parameter Generative Pre-trained Transformer (GPT) model with 24 layers, hidden size 2048, FFN size 8192, 16 attention heads, and sequence length 2048, in BF16 with data parallelism, using weak scaling. In weak scaling, which follows Gustafson’s law [26], the work per GPU is held fixed as GPUs are added; here the per-GPU batch size stays constant, so the global batch grows with the GPU count. Table V reports B200 and RTX PRO 6000 throughput and gradient-synchronization cost.

TABLE V
SINGLE-NODE MEGATRON-LM WEAK-SCALING ON B200 AND RTX PRO 6000. THE GRADS-SYNC COLUMN IS THE PER-ITERATION GRADIENT ALL-REDUCE TIME, IN MS AND AS A PERCENTAGE OF ITERATION TIME; EFFICIENCY IS RELATIVE TO THE 1-GPU THROUGHPUT.

GPU	GPUs /node	TFLOP/s /GPU	grads-sync (ms / %)	eff. %
B200	1	1031.6	5.6 / 0.05	100.0
	2	1005.3	66 / 0.59	97.5
	4	993.5	79 / 0.70	96.3
	8	986.0	84 / 0.74	95.6
RTX PRO 6000	1	281.5	4.0 / 0.05	100.0
	2	276.2	158 / 1.96	98.1
	4	267.6	360 / 4.34	95.1
	8	265.3	435 / 5.20	94.2

Intra-node weak scaling is excellent: B200 throughput holds at 95.6% of the single-GPU rate across eight GPUs, and gradient synchronization stays under 0.8% of step time; the fast NVLink fabric keeps the communication cost almost invisible.

Multi-node behavior is reported separately in Fig. 4, which shows both the gradient-synchronization cost and the weak-scaling efficiency as a function of the number of GPUs per node k . As k rises from 1 to 8 at fixed node count, gradient-synchronization cost falls from 1033 ms to 141 ms, nearly $1/k$. NCCL uses a hierarchical All-Reduce [19] that performs an intra-node Reduce-Scatter, then an inter-node All-Reduce, and finally an intra-node All-Gather, so with more GPUs per node the fast NVLink fabric handles most of the reduction and only a $1/k$ slice crosses the slower InfiniBand fabric. Weak-scaling efficiency (Fig. 4, line) is measured relative to the matching single-node baseline at the same GPUs per node, so that it isolates only the added inter-node cost. It is the mirror image of the gradient-sync cost, rising from 90.4% at one GPU per node to 98.9% at eight as the shrinking inter-node slice leaves less overhead. Packing GPUs densely per node therefore strongly reduces proportional communication overhead. We note SHARP was not enabled in these runs; given Section IV, enabling it is the highest-impact change available for multi-node data-parallel training.

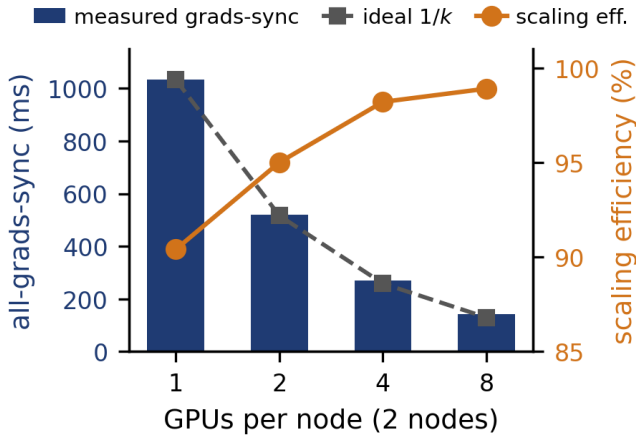


Fig. 4. Two-node B200 weak-scaling versus GPUs per node k : measured gradient-sync cost (bars), the ideal $1/k$ curve (dashed, anchored at $k=1$), and scaling efficiency (line).

The same runs on RTX PRO 6000 nodes, in the lower block of Table V, highlight the fabric gap. Per-GPU BF16 training throughput on B200 is $3.7\times$ that of the RTX PRO 6000 (1031.6 versus 281.5 TFLOP/s), matching the compute microbenchmark ratio (Fig. 1). Over 1 to 8 GPUs in a single node the RTX gradient-synchronization cost climbs to about 5.2% of step time versus under 0.8% on B200, a roughly $7\times$ larger relative overhead. This reflects the NVLink-versus-PCIe difference of Section IV, but it is much smaller than the $18\text{--}64\times$ gap between the two fabrics there. The reason is message size: gradient All-Reduces send small bucketed messages,

much smaller than the 16 GB messages of Table III. At those small sizes B200 is latency-bound and runs far below its 841 GB/s NVLink peak, while RTX already sits near its ≈ 13 GB/s Infinity-Fabric floor, so the two fabrics deliver far closer effective bandwidths than their peaks suggest. Consequently, RTX PRO 6000 still sustains a healthy weak-scaling efficiency of about 94% across eight GPUs in a node, despite lacking NVLink.

Comparing training throughput to the GEMM microbenchmark on the same silicon, Megatron reaches 69% of the gpu-fryer BF16 number on B200 (1031.6/1493) and 67% on the RTX PRO 6000 (281.5/419). This gap is expected rather than a misconfiguration. Table VI itemizes where wall-clock step time goes: only the GEMMs contribute to the model-FLOP numerator, and even those run at lower Tensor-Core utilization than the microbenchmark’s large square tile because a 1.3 B model has smaller GEMM shapes. Everything else consumes time in the denominator while adding nothing to the numerator: memory-bound normalization and element-wise kernels, the Adam optimizer step [27], gradient communication, and launch overhead. On B200 this corresponds to about 46% MFU against the 2250 TFLOP/s dense BF16 peak (Table II), at the high end of the 30–50% range typical of transformer pretraining [11], [14]. The near-identical 67–69% ratio across two very different Blackwell dies indicates the end-to-end overhead profile scales proportionally with raw GEMM throughput.

TABLE VI
WHERE THE TRAINING WALL-CLOCK STEP TIME GOES

Cost in step time	FLOP-counted?
Forward/backward GEMMs (varied, some small)	yes, lower Tensor-Core utilization
LayerNorm, softmax, dropout, residuals	no, memory-bound
Adam step (BF16 gradient $\rightarrow$ FP32 master weights)	no, memory-bound
Gradient All-Reduce	no, communication
Kernel launch / Python / scheduling	no

VI. DISCUSSION AND CONCLUSION

We have presented a layered, hardware-versus-software characterization of the AICR Blackwell cluster across compute, interconnect, and end-to-end training, on both its GPU types. The central finding is that B200 compute and intra-node communication run at their hardware ceilings, so the recoverable performance is algorithmic: SHARP bypasses the inter-node bidirectional wall for a $2.2\times$ All-Reduce speedup. The two GPU types serve complementary roles: the NVLink-equipped B200 is the platform for scaled multi-GPU training, whereas the RTX PRO 6000 nodes are best used for single- or two-GPU work. We summarize the per-layer binding constraints in Table VII and the resulting parallelism guidance in Table VIII.

Table VII asks one question of each measurement: is the limit set by hardware or by software? Its three blocks separate intra-node, inter-node, and end-to-end training results. Intra-node, the limit is hardware throughput: B200 compute and communication already run close to their ceilings, while the

non-NVLink RTX PRO 6000 communicates well with up to two GPUs but is limited beyond that. Inter-node, Send-Recv saturates a hardware wall the two GPU types share, set by the PCIe DMA engine splitting its budget between transmit and receive, whereas the remaining gaps are software and therefore recoverable: SHARP already recovers the Ring All-Reduce penalty, and All-to-All awaits a better-pipelined implementation.

End-to-end training scales efficiently within and across nodes on both GPU types (Table V and Fig. 4). Its GEMMs run at lower Tensor-Core utilization than the microbenchmark tile and the non-GEMM work of Table VI adds step time without adding FLOPs, giving about 46% MFU on B200, at the high end of the typical range for transformer pretraining; the gap is expected to narrow with FP8 and larger GEMM shapes.

TABLE VII
BINDING CONSTRAINT PER LAYER

Layer / operation	Achieved	Limit
Compute FP8 (B200)	91% of peak	hardware
Intra-node All-Reduce (B200)	93% of NVLink	hardware
Intra-node coll. (RTX, 4 GPU)	21% of PCIe	hardware (IF)
Inter-node Send-Recv (B200)	~100%	hardware wall
Inter-node All-Reduce (Ring)	79% of wall	software
+ SHARP	2.2× Ring	recovered
Inter-node All-to-All	19%	software
Training vs. measured GEMM	69%	mixed
Training MFU (vs. dense peak)	46%	mixed

Table VIII distills the characterization into operational guidance for distributed deep-learning training. The recommendations are direct consequences of the binding constraints measured above: each parallelism strategy should be deployed so that its dominant collective stays on the fast side of the fabric boundaries. Data parallelism is the first choice whenever the model fits in one GPU’s memory, scaling near-perfectly within a node and well across nodes; when the model does not fit, tensor parallelism should stay within one B200 node with NVLink while pipeline parallelism may cross nodes. For MoE models, whose All-to-All dispatch is the most demanding collective [28], [29], the job should be confined to a single node when running with stock NCCL.

We plan to extend this characterization in the following directions. (1) *Larger models*: train bigger models, whose larger GEMM shapes should raise MFU. (2) *More nodes*: extend the sweeps beyond two nodes, where deeper SHARP trees and rail-optimized routing come into play. (3) *Combined parallelism*: run tensor and pipeline parallelism alongside data parallelism to exercise the remaining collectives. (4) *SHARP in Megatron-LM*: enable SHARP in production Megatron-LM runs and quantify its end-to-end training speedup. (5) *FP8 training*: train in FP8 [30] via the Transformer Engine library [31], roughly doubling the compute ceiling of the GEMMs. (6) *Storage I/O*: characterize the parallel filesystem under checkpoint and data-loading workloads to complete the

TABLE VIII
OPERATIONAL GUIDANCE FOR DISTRIBUTED DEEP-LEARNING TRAINING,
BY PARALLELISM STRATEGY AND NODE TYPE

Scenario	Recommendation
B200 nodes	
Data parallel	First choice. Intra-node: gradient-sync cost on NVLink is minimal. Inter-node: enable SHARP (improves gradient-sync).
Tensor parallel	Activation All-Reduces in every transformer layer need the highest bandwidth: keep within one B200 node with NVLink.
Pipeline parallel	Stage-boundary Send-Recv may cross nodes; budget bubbles for the inter-node bandwidth wall.
MoE	All-to-All is the weakest inter-node collective: with stock NCCL, confine the job to one node.
RTX PRO 6000 nodes	
All workloads	Single- or two-GPU training, inference, and development only.

full-stack picture.

ACKNOWLEDGMENT

The authors acknowledge the MGHPCC AI Computing Resource (AICR), with support from the Commonwealth of Massachusetts and the AICR participating institutions, for providing AI and high-performance computing resources used in the results reported in this publication. We thank Brian Gregor (Boston University), Aya Nawano (Yale University), Will Paik (Northeastern University), and Paula Sanematsu (Harvard University), of the AICR benchmarking group, for helpful discussions.

REFERENCES

- [1] MGHPCC, “AICR system description.” [Online]. Available: <https://docs.aicr.ai/system-description/>
- [2] NVIDIA, “NVIDIA Blackwell architecture technical brief,” 2024.
- [3] NVIDIA, “NVIDIA Quantum-2 InfiniBand platform.” [Online]. Available: <https://www.nvidia.com/en-us/networking/quantum2/>
- [4] NVIDIA, “NVIDIA H100 Tensor Core GPU architecture,” Technical Whitepaper, 2022.
- [5] A. Jarmusch et al., “Microbenchmarking NVIDIA’s Blackwell architecture: An in-depth architectural analysis,” arXiv:2512.02189, 2025.
- [6] S. Williams, A. Waterman, and D. Patterson, “Roofline: an insightful visual performance model for multicore architectures,” *Commun. ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [7] A. Li et al., “Evaluating modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 1, pp. 94–110, 2020.
- [8] NVIDIA, “NCCL: Optimized primitives for collective multi-GPU communication.” [Online]. Available: <https://github.com/NVIDIA/nccl>
- [9] R. L. Graham et al., “Scalable Hierarchical Aggregation Protocol (SHaRP): A hardware architecture for efficient data reduction,” in *Proc. Workshop on Communication Optimizations in HPC (COMHPC)*, 2016, pp. 1–10.
- [10] P. Mattson et al., “MLPerf training benchmark,” in *Proc. Mach. Learn. Syst. (MLSys)*, 2020.
- [11] A. Chowdhery et al., “PaLM: Scaling language modeling with pathways,” *J. Mach. Learn. Res.*, vol. 24, pp. 1–113, 2023.
- [12] M. Shoeny et al., “Megatron-LM: Training multi-billion parameter language models using model parallelism,” arXiv:1909.08053, 2019.
- [13] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.
- [14] D. Narayanan et al., “Efficient large-scale language model training on GPU clusters using Megatron-LM,” in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, 2021.

- [15] NVIDIA, “NVIDIA DGX SuperPOD reference architecture,” Technical Whitepaper, 2023.
- [16] “gpu-fryer: GPU stress test and health-check tool.” [Online]. Available: <https://github.com/huggingface/gpu-fryer>
- [17] NVIDIA, “NCCL Tests.” [Online]. Available: <https://github.com/NVIDIA/nccl-tests>
- [18] NVIDIA, “NVIDIA A100 Tensor Core GPU architecture,” Technical Whitepaper, 2020.
- [19] Z. Hu et al., “Demystifying NCCL: An in-depth analysis of GPU communication protocols and algorithms,” arXiv:2507.04786, 2025.
- [20] NVIDIA, “GPUDirect RDMA.” [Online]. Available: <https://docs.nvidia.com/cuda/gpudirect-rdma/>
- [21] P. Patarasuk and X. Yuan, “Bandwidth optimal all-reduce algorithms for clusters of workstations,” *J. Parallel Distrib. Comput.*, vol. 69, no. 2, pp. 117–124, 2009.
- [22] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *Int. J. High Perform. Comput. Appl.*, vol. 19, no. 1, pp. 49–66, 2005.
- [23] J. Bruck, C.-T. Ho, S. Kipnis, E. Upfal, and D. Weathersby, “Efficient algorithms for all-to-all communications in multiport message-passing systems,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 8, no. 11, pp. 1143–1156, 1997.
- [24] DeepSeek-AI, “DeepEP: An efficient expert-parallel communication library.” [Online]. Available: <https://github.com/deepseek-ai/DeepEP>
- [25] PCI-SIG, “PCI Express Base Specification, Revision 5.0, Version 1.0,” 2019.
- [26] J. L. Gustafson, “Reevaluating Amdahl’s law,” *Commun. ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [27] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [28] C. Hwang et al., “Tutel: Adaptive mixture-of-experts at scale,” in *Proc. Mach. Learn. Syst. (MLSys)*, 2023.
- [29] S. Rajbhandari et al., “DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022.
- [30] P. Micikevicius et al., “FP8 formats for deep learning,” arXiv:2209.05433, 2022.
- [31] NVIDIA, “Transformer Engine.” [Online]. Available: <https://github.com/NVIDIA/TransformerEngine>